

TP3 – Équations différentielles

HAMLIL Mohamed

2024-09-01

Table des matières

1	TP3 – Équations différentielles	1
1.1	6. Librairie R	1
1.2	7.1 L'équation	1
1.3	7.3 Champ de directions	1
1.4	7.4 Solution de (C)	2
1.5	7.5 Variations de	3
1.6	7.6 Solutions en fonction de	5
1.7	8.1 Modèle de Malthus	6
1.8	8.2 Modèle logistique de Verhulst	6
1.9	9. Travail à rendre	7
1.9.1	9.1 Pêche sans quota	7
1.9.2	9.2 Pêche par quota	8

1 TP3 – Équations différentielles

Ce notebook présente la résolution numérique d'équations différentielles en R à l'aide de la librairie **deSolve**.

1.1 6. Librairie R

Nous utilisons le package **deSolve** pour le traitement des équations différentielles.

```
# Installation du package (à exécuter une seule fois si nécessaire)
# install.packages("deSolve")

# Chargement de la librairie
library(deSolve)
```

1.2 7.1 L'équation

Nous étudions le système de Cauchy suivant :

Nous implémentons la fonction $f(t, y, a)$ correspondant à l'équation différentielle.

```
# Définition de la fonction f
f <- function(t, y, a) { a * y * (t - y) }
```

1.3 7.3 Champ de directions

Pour visualiser le comportement des solutions, nous traçons le champ de directions. La fonction **champ** ci-dessous crée une grille de points et trace des flèches dont la pente est donnée par l'équation différentielle.

```

# Fonction pour tracer le champ de directions
champ <- function(xmax, pas, f, a) {
  x <- c(-xmax, xmax)
  # Création d'un graphique vide avec les bonnes échelles
  plot(x, x, type = "n", xlab = "t", ylab = "y")

  # Discrétisation des axes
  discr <- seq(-xmax, xmax, pas)

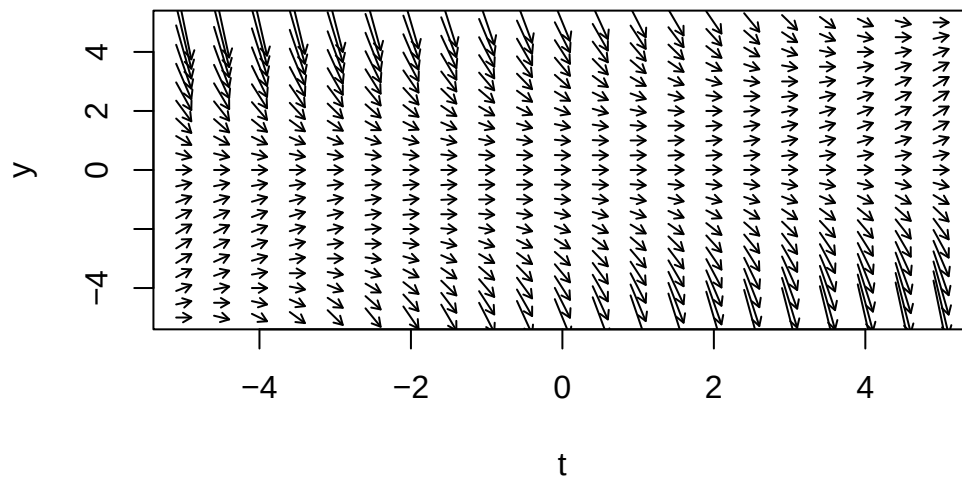
  # Création de la grille de points
  points <- expand.grid(discr, discr)

  # Calcul des pentes et tracé des flèches
  # On normalise la longueur des flèches pour la lisibilité
  arrows(
    points[, 1] - pas / 5,
    points[, 2] - pas / 5 * f(points[, 1], points[, 2], a),
    points[, 1] + pas / 5,
    points[, 2] + pas / 5 * f(points[, 1], points[, 2], a),
    length = 0.05
  )
}

# Exemple d'utilisation avec a = 0.25
a <- 0.25
champ(5, 0.5, f, a)
title(main = paste("Champ de directions pour a =", a))

```

Champ de directions pour a = 0.25



1.4 7.4 Solution de (C)

Nous utilisons la fonction `ode` pour résoudre numériquement l'équation. `deSolve` requiert que la fonction définissant l'équation renvoie une liste.

```

# Wrapper pour deSolve : la fonction doit renvoyer une liste
Eq <- function(t, y, a) { list(f(t, y, a)) }

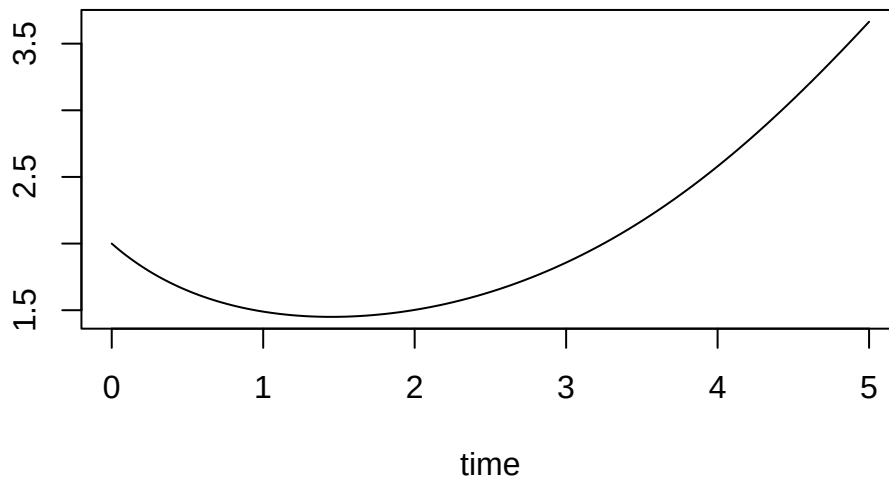
```

```
# Paramètres
y0 <- 2
a <- 0.25
discr <- seq(0, 5, 0.02) # Discrétisation du temps

# Résolution
sol <- ode(y = y0, times = discr, func = Eq, parms = a)

# Affichage de la solution
plot(sol, main = "Solution de (C)")
```

Solution de (C)



1.5 7.5 Variations de

Nous observons les trajectoires pour différentes conditions initiales sur le champ de vecteurs.

```
# Tracer le champ de directions
champ(5, 0.5, f, a)
title(main = "Solutions pour différentes conditions initiales")

# Résolution pour différentes valeurs initiales
y0_vals <- c(1, 2, -1)
colors <- c("red", "blue", "green")

for (i in 1:length(y0_vals)) {
  sol <- ode(y = y0_vals[i], times = discr, func = Eq, parms = a)
  lines(sol, col = colors[i], lwd = 2)
}
```

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 2.77711, R2 = 2.0845e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.

In above message, $R1 = 2.77711$, $R2 = 2.0845e-16$

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, $R1 = 2.77711$, $R2 = 1.72667e-16$

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, $R1 = 2.77711$, $R2 = 1.72667e-16$

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, $R1 = 2.77711$, $R2 = 1.72667e-16$

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, $R1 = 2.77711$, $R2 = 1.3803e-16$

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, $R1 = 2.77711$, $R2 = 1.3803e-16$

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, $R1 = 2.77711$, $R2 = 1.14335e-16$

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, $R1 = 2.77711$, $R2 = 1.14335e-16$

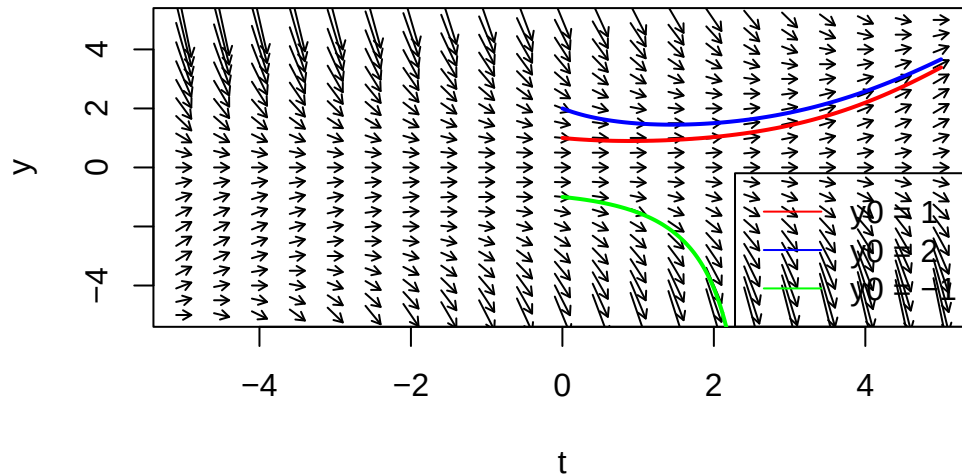
DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, $R1 = 2.77711$, $R2 = 1.14335e-16$

DLSODA- Above warning has been issued I1 times.
It will not be issued again for this problem.
In above message, $I1 = 10$

DLSODA- At T (=R1), too much accuracy requested
for precision of machine.. See TOLSF (=R2)
In above message, $R1 = 2.77711$, $R2 = \text{nan}$

```
legend("bottomright", legend = paste("y0 =", y0_vals), col = colors, lty = 1)
```

Solutions pour différentes conditions initiales



1.6 7.6 Solutions en fonction de

Nous fixons et faisons varier le paramètre .

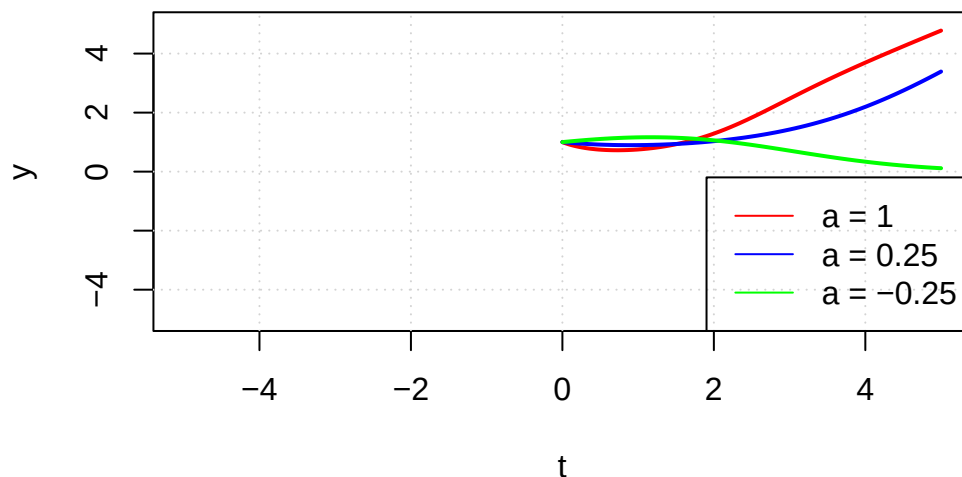
```
# Configuration de la fenêtre graphique
plot(c(-5, 5), c(-5, 5), type = "n", xlab = "t", ylab = "y", main = "Variations selon a")
grid()

y0 <- 1
a_vals <- c(1, 0.25, -0.25)
colors <- c("red", "blue", "green")

for (i in 1:length(a_vals)) {
  sol <- ode(y = y0, times = discr, func = Eq, parms = a_vals[i])
  lines(sol, col = colors[i], lwd = 2)
}

legend("bottomright", legend = paste("a =", a_vals), col = colors, lty = 1)
```

Variations selon a



1.7 8.1 Modèle de Malthus

Modèle de croissance exponentielle : .

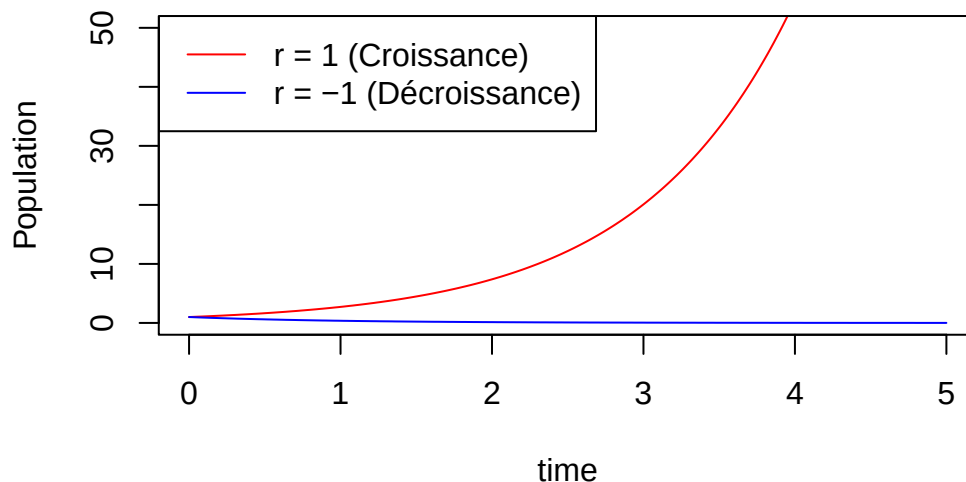
```
# Définition du modèle de Malthus
f_malthus <- function(t, y, r) { r * y }
Eq_malthus <- function(t, y, r) { list(f_malthus(t, y, r)) }

# Paramètres
y0 <- 1
discr <- seq(0, 5, 0.02)

# Résolution pour r = 1 et r = -1
sol1 <- ode(y = y0, times = discr, func = Eq_malthus, parms = 1)
sol2 <- ode(y = y0, times = discr, func = Eq_malthus, parms = -1)

# Tracé
plot(sol1, ylim = c(0, 50), col = "red", main = "Modèle de Malthus", ylab = "Population")
lines(sol2, col = "blue")
legend("topleft", legend = c("r = 1 (Croissance)", "r = -1 (Décroissance)"), col = c("red", "blue"), lty = c(1, 1))
```

Modèle de Malthus



1.8 8.2 Modèle logistique de Verhulst

Modèle avec capacité limite : .

```
# Définition du modèle de Verhulst
f_verhulst <- function(t, y, p) {
  r <- p[1]
  K <- p[2]
  r * y * (1 - y / K)
}
Eq_verhulst <- function(t, y, p) { list(f_verhulst(t, y, p)) }

# Paramètres (r, K)
p1 <- c(1.5, 2.5)
p2 <- c(-1.5, 2.5)
y0 <- 1
```

```

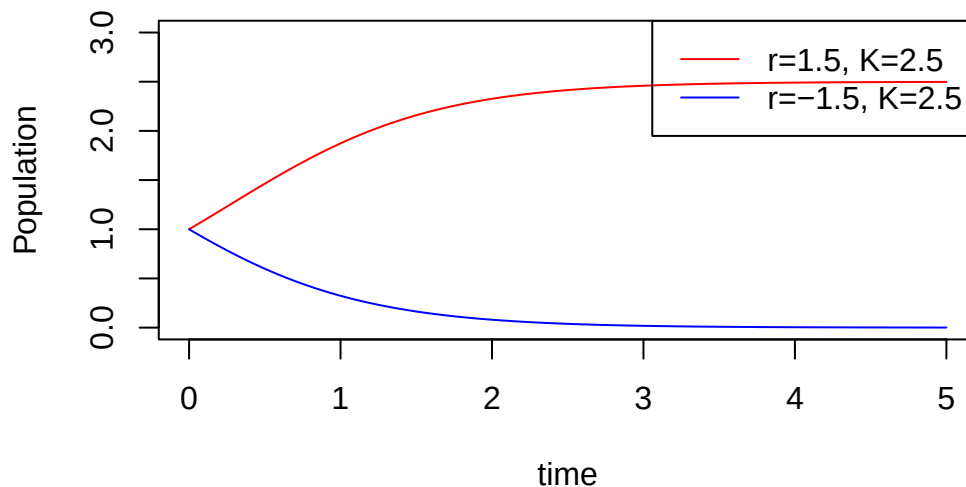
discr <- seq(0, 5, 0.02)

# Résolution
sol_v1 <- ode(y = y0, times = discr, func = Eq_verhulst, parms = p1)
sol_v2 <- ode(y = y0, times = discr, func = Eq_verhulst, parms = p2)

# Tracé
plot(sol_v1, ylim = c(0, 3), col = "red", main = "Modèle de Verhulst", ylab = "Population")
lines(sol_v2, col = "blue")
legend("topright", legend = c("r=1.5, K=2.5", "r=-1.5, K=2.5"), col = c("red", "blue"), lty = 1)

```

Modèle de Verhulst



1.9 9. Travail à rendre

1.9.1 9.1 Pêche sans quota

Modèle : .

```

# Définition du modèle
f_peche_sans <- function(t, y, params) {
  r <- params["r"]
  K <- params["K"]
  E <- params["E"]
  r * y * (1 - y / K) - E * y^2
}

Eq_peche_sans <- function(t, y, params) { list(f_peche_sans(t, y, params)) }

# Paramètres fixes
y0 <- 1
K <- 5
r <- 0.08 # Choix arbitraire < 0.1

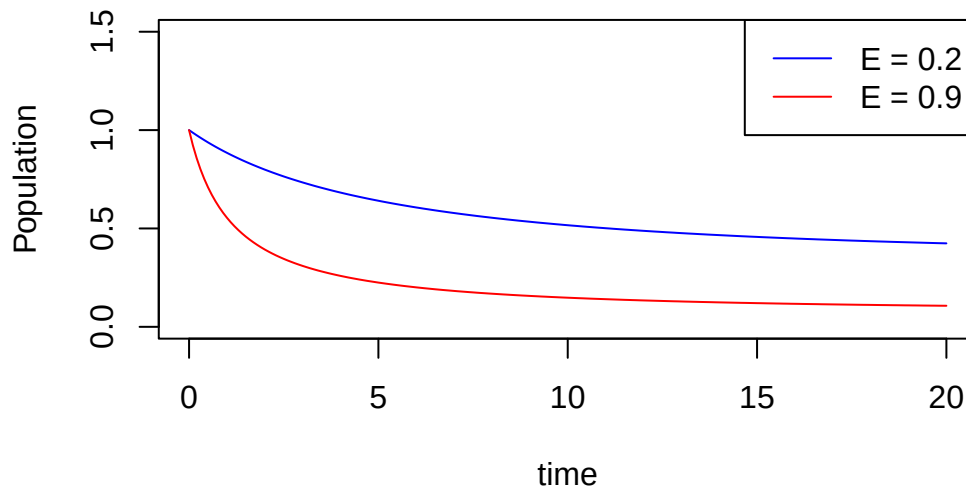
# Effort de pêche E1 = 0.2
params1 <- c(r = r, K = K, E = 0.2)
# Effort de pêche E2 = 0.9
params2 <- c(r = r, K = K, E = 0.9)

```

```
# Résolution
discr <- seq(0, 20, 0.1)
sol_e1 <- ode(y = y0, times = discr, func = Eq_peche_sans, parms = params1)
sol_e2 <- ode(y = y0, times = discr, func = Eq_peche_sans, parms = params2)

# Tracé des solutions
plot(sol_e1, type = "l", col = "blue", ylim = c(0, 1.5), main = "Pêche sans quota", ylab = "Population")
lines(sol_e2, col = "red")
legend("topright", legend = c("E = 0.2", "E = 0.9"), col = c("blue", "red"), lty = 1)
```

Pêche sans quota



1.9.2 9.2 Pêche par quota

Modèle :

```
# Définition du modèle
f_peche_quota <- function(t, y, params) {
  r <- params["r"]
  K <- params["K"]
  Q <- params["Q"]
  r * y * (1 - y / K) - Q
}

Eq_peche_quota <- function(t, y, params) { list(f_peche_quota(t, y, params)) }

# Paramètres
r <- 0.5 # Choix arbitraire entre 0.4 et 0.6
K <- 5
y0 <- 1

# Quotas
Q1 <- 0.8 # Population devrait diminuer
Q2 <- 0.1 # Population devrait augmenter

paramsQ1 <- c(r = r, K = K, Q = Q1)
paramsQ2 <- c(r = r, K = K, Q = Q2)

# Résolution
```



```
sol_q1 <- ode(y = y0, times = discr, func = Eq_peche_quota, parms = paramsQ1)
```

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 3.75373e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 3.75373e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 3.10936e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 3.10936e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 3.10936e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 2.48562e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 2.48562e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 2.05893e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 2.05893e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 2.05893e-16

DLSODA- Above warning has been issued 11 times.
It will not be issued again for this problem.

In above message, I1 = 10

DLSODA- At T (=R1), too much accuracy requested
for precision of machine.. See TOLSF (=R2)

In above message, R1 = 5.46334, R2 = nan

```
sol_q2 <- ode(y = y0, times = discr, func = Eq_peche_quota, parms = paramsQ2)

# Tracé
plot(sol_q1, type = "l", col = "orange", ylim = c(-1, 5), main = "Pêche par quota", ylab = "Population")
lines(sol_q2, col = "purple")
abline(h = 0, col = "gray", lty = 2)
legend("topright", legend = c(paste("Q =", Q1), paste("Q =", Q2)), col = c("orange", "purple"), lty = 1)
```

